

Assignment 5: Data Visualization

Shuer Li

Fall 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Visualization

Directions

1. Rename this file `<FirstLast>_A05_DataVisualization.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up your session

1. Set up your session. Load the tidyverse, here & cowplot packages, and verify your home directory. Read in the NTL-LTER processed data files for nutrients and chemistry/physics for Peter and Paul Lakes (use the tidy NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv version in the Processed_KEY folder) and the processed data file for the Niwot Ridge litter dataset (use the NEON_NIWO_Litter_mass_trap_Processed.csv version, again from the Processed_KEY folder).
2. Make sure R is reading dates as date format; if not change the format to date.

```
#1 Load necessary packages  
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
## v dplyr      1.1.4      v readr      2.1.5  
## v forcats    1.0.0      v stringr   1.5.1  
## v ggplot2    4.0.0      v tibble    3.2.1  
## v lubridate  1.9.4      v tidyr     1.3.1  
## v purrr      1.0.2  
## -- Conflicts ----- tidyverse_conflicts() --  
## x dplyr::filter() masks stats::filter()  
## x dplyr::lag()     masks stats::lag()  
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(here)
```

```
## here() starts at /home/guest/EDE_Fall2025
```

```
library(cowplot)
```

```
##  
## Attaching package: 'cowplot'  
##  
## The following object is masked from 'package:lubridate':  
##  
##     stamp
```

```
# Verify the working directory  
here()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

```
# Define the processed data folder  
processed_data <- here("Data/Processed_KEY")
```

```
# Read the Peter and Paul Lakes data  
PeterPaul.chem.nutrients <- read.csv(  
  here(processed_data, "NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv"),  
  stringsAsFactors = TRUE)
```

```
# Read the Niwot Ridge litter dataset  
NIWO.litter <- read.csv(  
  here(processed_data, "NEON_NIWO_Litter_mass_trap_Processed.csv"),  
  stringsAsFactors = TRUE)
```

```
#2 Check which columns are dates  
names(PeterPaul.chem.nutrients)
```

```
## [1] "lakename"      "year4"          "daynum"          "month"  
## [5] "sampledate"   "depth"          "temperature_C"   "dissolvedOxygen"  
## [9] "irradianceWater" "irradianceDeck" "tn_ug"           "tp_ug"  
## [13] "nh34"          "no23"           "po4"
```

```
names(NIWO.litter)
```

```
## [1] "plotID"        "trapID"         "collectDate"     "functionalGroup"  
## [5] "dryMass"       "qaDryMass"      "subplotID"       "decimalLatitude"  
## [9] "decimalLongitude" "elevation"      "nlcdClass"       "plotType"  
## [13] "geodeticDatum"
```

```
# Convert date columns to proper Date format  
PeterPaul.chem.nutrients$sampledate <- ymd(PeterPaul.chem.nutrients$sampledate)  
NIWO.litter$collectDate <- ymd(NIWO.litter$collectDate)
```

```
# Verify that conversion was successful  
str(PeterPaul.chem.nutrients$sampledate)
```

```
## Date[1:23008], format: "1984-05-27" "1984-05-27" "1984-05-27" "1984-05-27" "1984-05-27" ...
```

```
str(NIW0.litter$collectDate)
```

```
## Date[1:1692], format: "2016-06-16" "2016-06-16" "2016-06-16" "2016-06-16" "2016-06-16" ...
```

Define your theme

3. Build a theme and set it as your default theme. Customize the look of at least two of the following:

- Plot background
- Plot title
- Axis labels
- Axis ticks/gridlines
- Legend

```
#3 Create a custom theme
```

```
library(ggthemes)
```

```
##
```

```
## Attaching package: 'ggthemes'
```

```
## The following object is masked from 'package:cowplot':
```

```
##
```

```
## theme_map
```

```
mytheme <- theme_base() +  
  theme(  
    # Line color and width  
    line = element_line(  
      color = "red",  
      linewidth = 1.2  
    ),  
  
    # Plot background  
    plot.background = element_rect(  
      fill = "white",  
      color = "gray85"  
    ),  
  
    # Plot title  
    plot.title = element_text(  
      face = "bold",  
      size = 15,  
      hjust = 0.5,  
      color = "black"  
    ),  
  
    # Axis labels and text  
    axis.title = element_text(  
      face = "bold",
```

```

    size = 13,
    color = "black"
  ),
  axis.text = element_text(
    size = 11,
    color = "black"
  ),

  # Axis gridlines
  panel.grid.major = element_line(
    color = "gray90",
    linewidth = 0.4
  ),
  panel.grid.minor = element_blank(),

  # Legend background and title
  legend.background = element_rect(
    color = "gray60",
    fill = "lightyellow"
  ),
  legend.title = element_text(
    color = "blue",
    face = "bold",
    size = 12
  ),
  legend.text = element_text(
    size = 11
  ),

  # Legend position
  legend.position = "top"
)

# Set as the default theme for all plots
theme_set(mytheme)

```

Create graphs

For numbers 4-7, create ggplot graphs and adjust aesthetics to follow best practices for data visualization. Ensure your theme, color palettes, axes, and additional aesthetics are edited accordingly.

4. [NTL-LTER] Plot total phosphorus (tp_ug) by phosphate (po4), with separate aesthetics for Peter and Paul lakes. Add line(s) of best fit using the `lm` method. Adjust your axes to hide extreme values (hint: change the limits using `xlim()` and/or `ylim()`).

```

#4 Plot total phosphorus by phosphate
# Add linear regression lines and adjust axis limits to hide outliers
library(viridis)

```

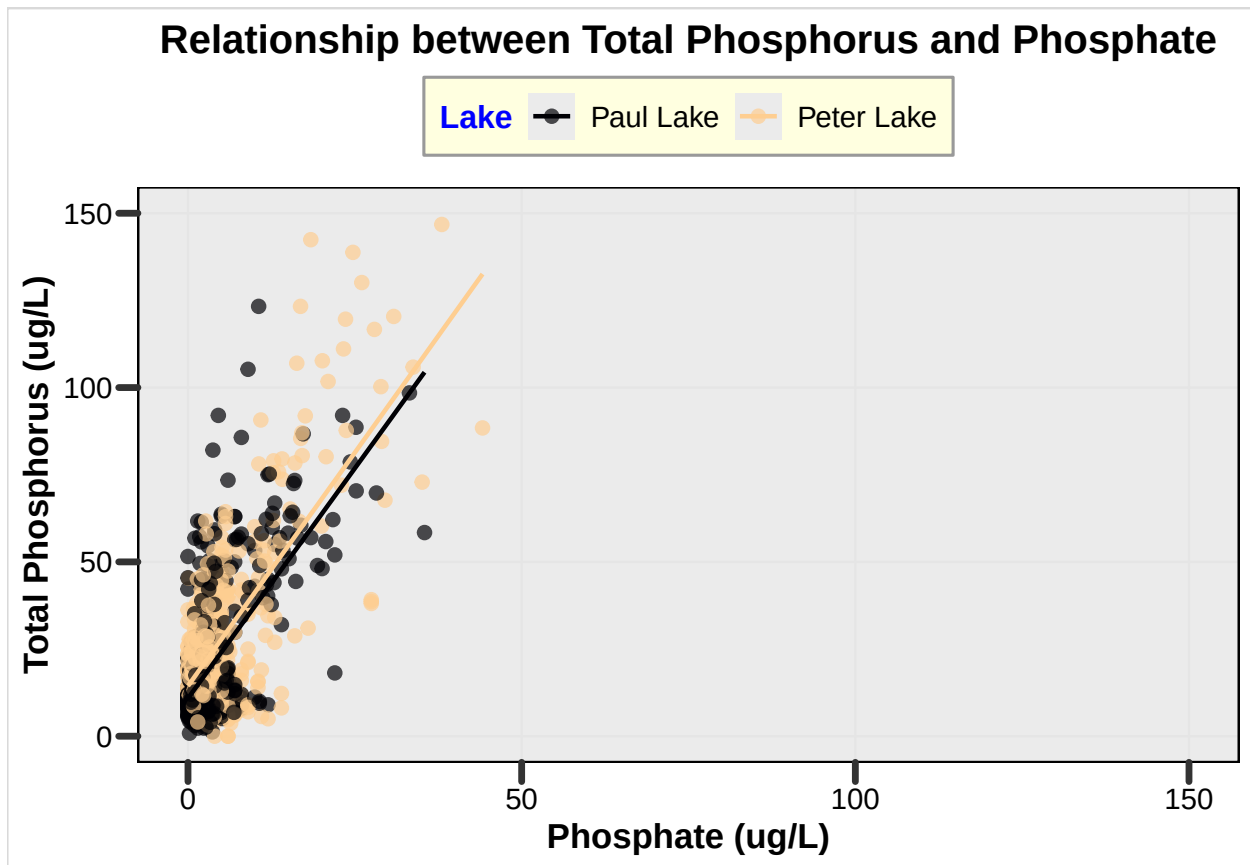
```
## Loading required package: viridisLite
```

```
ggplot(PeterPaul.chem.nutrients,
       aes(x = po4, y = tp_ug, color = lakename)) +
  geom_point(alpha = 0.7, size = 2) + # scatter points with transparency
  geom_smooth(method = "lm", se = FALSE, linewidth = 0.8) + # linear regression lines
  scale_color_viridis(discrete = TRUE, option = "magma", end = 0.9) + # color palette
  xlim(0, 150) + ylim(0, 150) + # limit axes to remove outliers
  labs(
    title = "Relationship between Total Phosphorus and Phosphate",
    x = "Phosphate (ug/L)",
    y = "Total Phosphorus (ug/L)",
    color = "Lake"
  )
)
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: Removed 21948 rows containing non-finite outside the scale range
## ('stat_smooth()').
```

```
## Warning: Removed 21948 rows containing missing values or values outside the scale range
## ('geom_point()').
```



5. [NTL-LTER] Make three separate boxplots of (a) temperature, (b) TP, and (c) TN, with month as the x axis and lake as a color aesthetic. Then, create a cowplot that combines the three graphs. Make sure that only one legend is present and that graph axes are aligned.

Tips: * Recall the discussion on factors in the lab section as it may be helpful here. * Setting an axis title in your theme to `element_blank()` removes the axis title (useful when multiple, aligned plots use the same axis values) * Setting a legend's position to "none" will remove the legend from a plot. * Individual plots can have different sizes when combined using `cowplot`.

```
#5 Create separate boxplots of temperature, TP, and TN by month and lake, then combine with cowplot
library(cowplot)

# Convert month to factor for proper ordering
PeterPaul.chem.nutrients$month <- as.factor(PeterPaul.chem.nutrients$month)

# (a) Temperature
p_temp <- ggplot(PeterPaul.chem.nutrients,
                 aes(x = month, y = temperature_C, color = lakename)) +
  geom_boxplot() +
  labs(y = "Temperature (deg C)", x = "Month") +
  theme(legend.position = "none")

# (b) Total Phosphorus (TP)
p_tp <- ggplot(PeterPaul.chem.nutrients,
               aes(x = month, y = tp_ug, color = lakename)) +
  geom_boxplot() +
  labs(y = "Total Phosphorus (ug/L)", x = "Month") +
  theme(legend.position = "none")

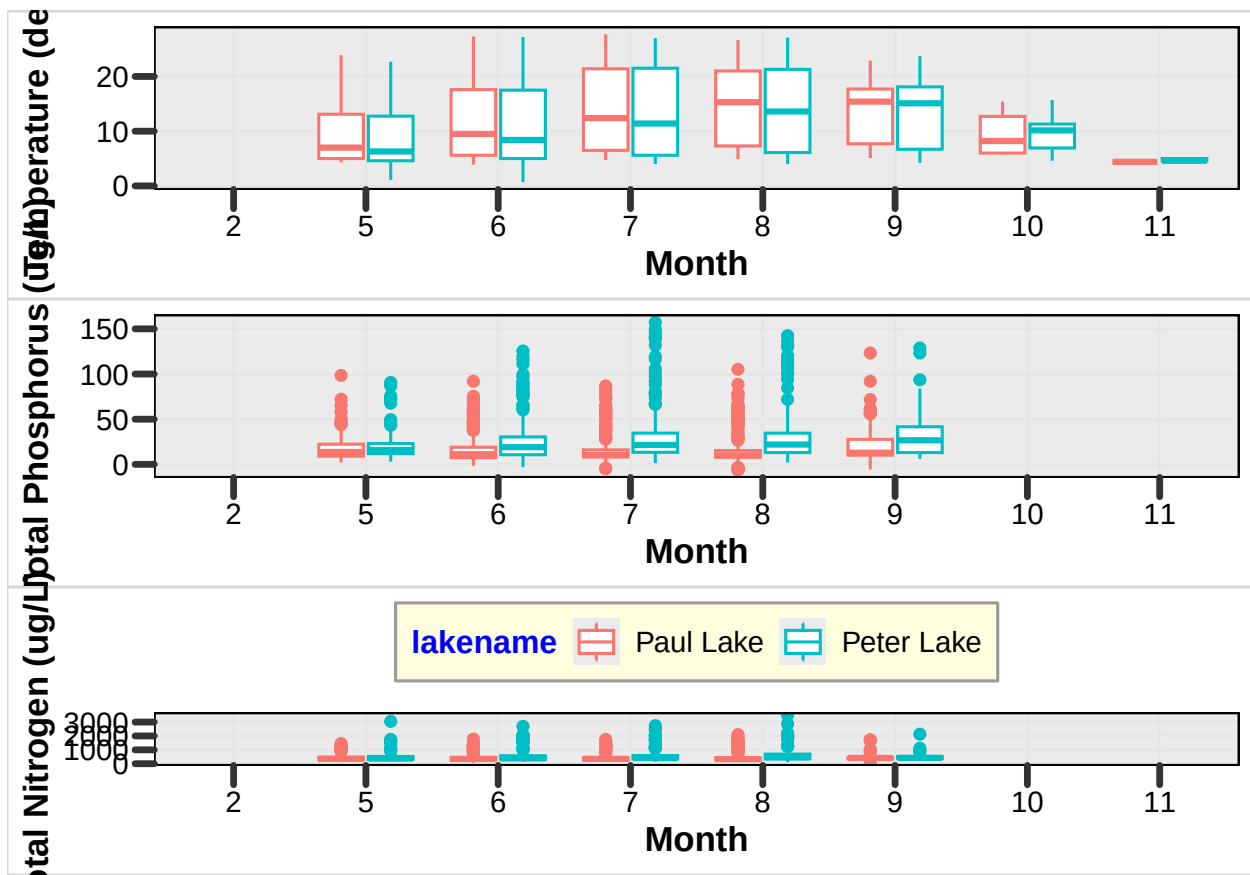
# (c) Total Nitrogen (TN)
p_tn <- ggplot(PeterPaul.chem.nutrients,
               aes(x = month, y = tn_ug, color = lakename)) +
  geom_boxplot() +
  labs(y = "Total Nitrogen (ug/L)", x = "Month") +
  theme(legend.position = "top")

# Combine plots and align axes
plot_grid(p_temp, p_tp, p_tn,
          ncol = 1, align = "v")
```

```
## Warning: Removed 3566 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

```
## Warning: Removed 20729 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

```
## Warning: Removed 21583 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

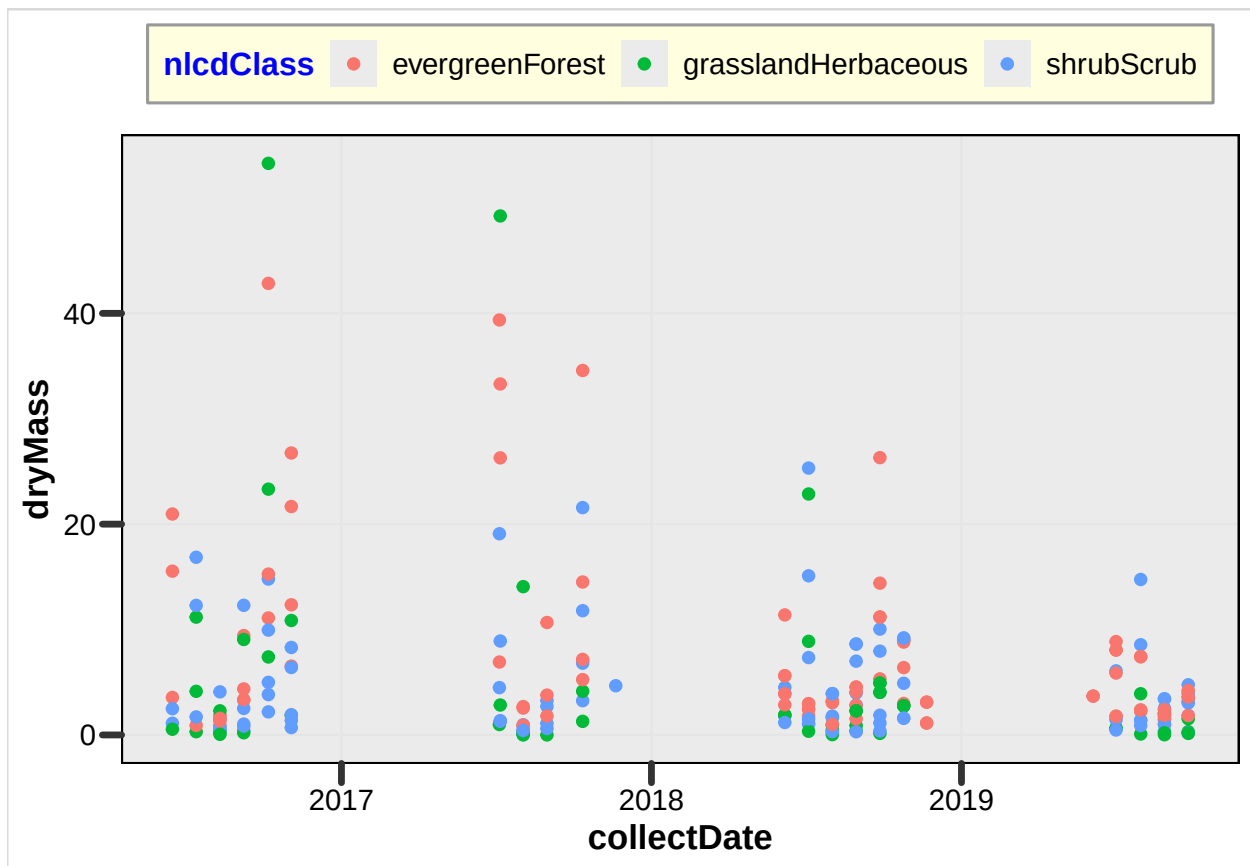


Question: What do you observe about the variables of interest over seasons and between lakes?

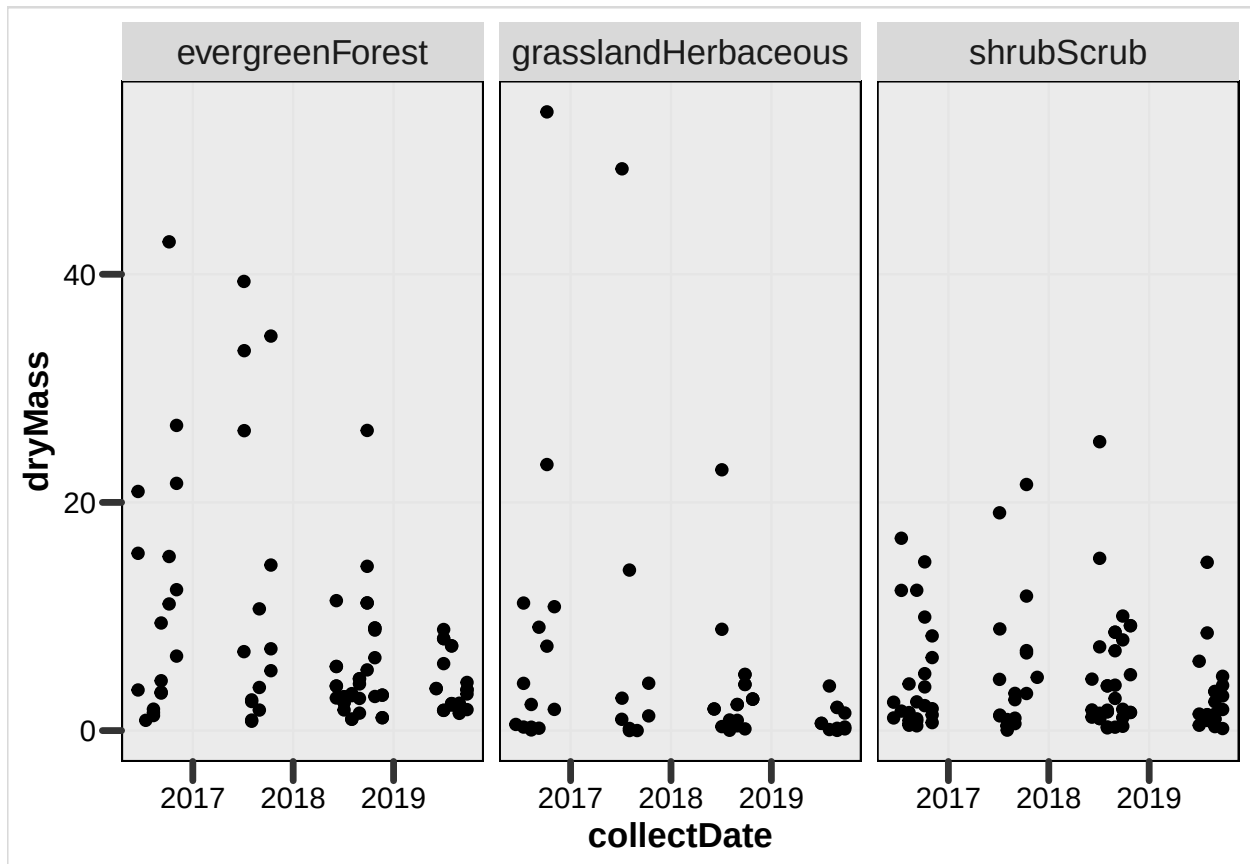
Answer: Temperature increases through the summer months and declines toward autumn, showing a clear seasonal pattern. In contrast, total phosphorus and total nitrogen vary less strongly with month but show slightly higher concentrations in Peter Lake compared to Paul Lake across most of the year. Overall, Peter Lake tends to have warmer and more nutrient-rich conditions.

6. [Niwot Ridge] Plot a subset of the litter dataset by displaying only the “Needles” functional group. Plot the dry mass of needle litter by date and separate by NLCD class with a color aesthetic. (no need to adjust the name of each land use)
7. [Niwot Ridge] Now, plot the same plot but with NLCD classes separated into three facets rather than separated by color.

```
#6 Plot dry mass of needle litter by date, colored by NLCD class
Needles <- subset(NIW0.litter, functionalGroup == "Needles")
ggplot(Needles, aes(x = collectDate, y = dryMass, color = nlcdClass)) +
  geom_point()
```



```
#7 Plot dry mass of needle litter by date, faceted by NLCD class  
ggplot(Needles, aes(x = collectDate, y = dryMass)) +  
  geom_point() +  
  facet_wrap(~ nlcdClass)
```

Question: Which of these plots (6 vs. 7) do you think is more effective, and why?

Answer: The faceted plot (7) is more effective because it separates each NLCD class into its own panel, making it easier to compare trends within and between land-cover types without color overlap. In contrast, the colored plot (6) is harder to interpret when points overlap across classes.